

Introduction

Goal: Realizzazione in Java del [GAME OF LIFE DI CONWAY](#) con GUI del gioco realizzata come una pagina Web.

La logica del gioco è già stata analizzata nello [Sprint1](#). Utiliziamo quindi il risultato ottenuto, che costituisce un sottosistema funzionante, come punto di partenza per questo nuovo lavoro (deployed in un file **conway26Java-1.0.jar**).

Requirements

Ricordiamo dai requisiti dello [Sprint1](#), che l’utente umano deve poter:

- specificare la configurazione iniziale della griglia del gioco
- vedere l’evoluzione del gioco in forma opportuna (si veda [Problema della vista del gioco](#))
- fermare e far ripartire l’evoluzione del gioco
- pulire (a gioco fermo) la configurazione della griglia del gioco

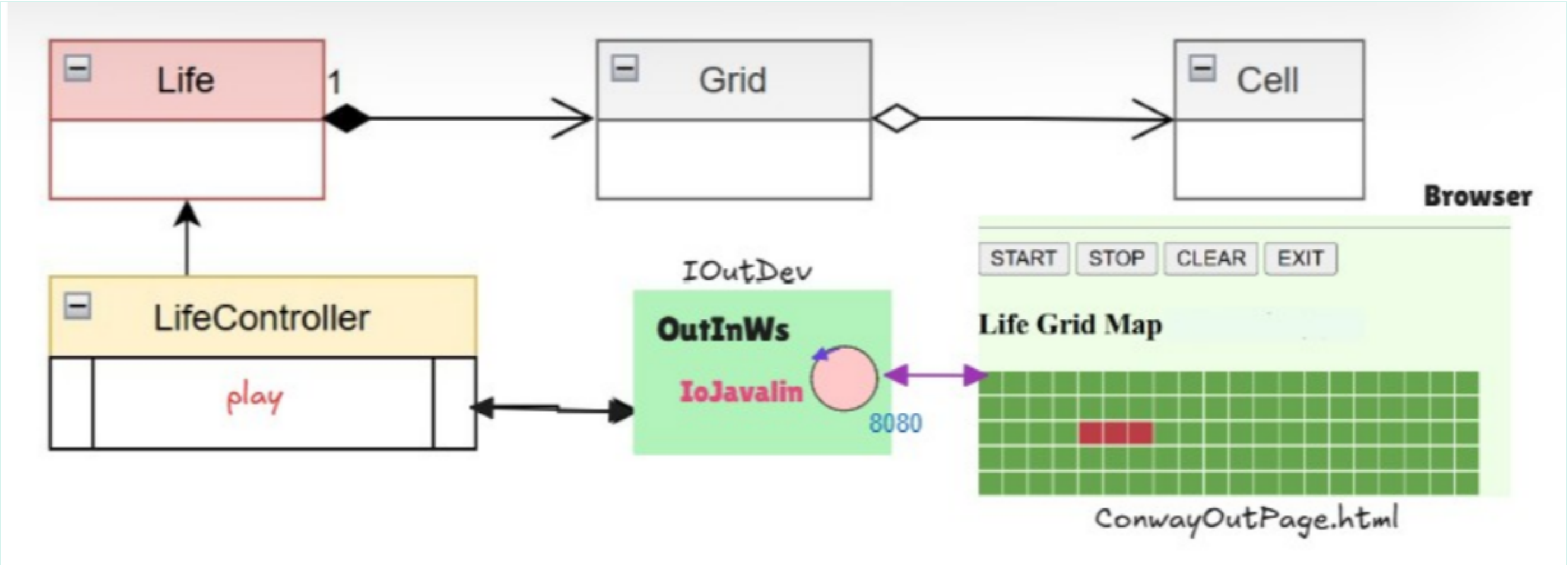
Ora si aggiungo i seguenti requisiti:

1. dotare il gioco Life di una pagina HTML come dispositivo di I/O
2. la pagina deve costituire un componente interno alla applicazione secondo la architettura riportata in [IoJavalin integrato](#)
3. il gestore del gioco sarà l’utente che ha aperto per primo (owner) una pagina HTML collegata al gioco. In altre parole, solo la pagina dell’owner avrà pulsanti di comando START/STOP/CLEAN/EXIT attivi
4. la pagina HTML deve essere aggiornata in modo automatico man mano il gioco procede
5. un utente non owner che si collega mentre il gioco è in corso, dovrebbe vedere lo stato attuale della griglia in modo corretto
6. il deployment del gioco deve avvenire mediante Docker

Requirement analysis

Il requisisito di dotare il gioco di una pagina HTML si inquadra nel contesto dei moderni sistemi Web, che vengono supportati da appositi servere da Browser.

Analizzo l'architettura del sistema che voglio realizzare (javalin integrato nell'applicazione), riportata in questo modello informale:



Il framework javalin costituisce il ‘cuore’ di un nuovo dispositivo di I/O, dando all’applicazione la responsabilità di creare la pagina HTML.

A livello strutturale, oltre alle classi già realizzate, noto che OutInWs dovrà implementare l'interfaccia [IOutDev](#) già definita e incapsulare il server IoJavalin per comunicare con la [pagina html](#) via browser. La funzione di questa classe è di fare da mediatore tra il LifeController che si occupa della gestione dell'evoluzione del gioco e il browser che si occupa di presentare all'utente una rappresentazione grafica del gioco. Sarà necessario un metodo *setController* per andare ad iniettare dentro a OutInWs un riferimento al LifeController.

Il lancio dell'applicazione du un computer permette di attivare un browser sullo stesso computer e digitare localhost:8080 (puntando il browser verso l’indirizzo di loopback).

Problem analysis

A livello di interazione, è necessario andare a definire un protocollo di interazione tra OutInWs e la pagina web, in modo che i messaggi utilizzino lo stesso linguaggio e che quindi entrambi possano essere in grado di interpretare correttamente il payload. Utilizzerò dei messaggi che seguono la struttura dettata da [IApplMessage](#), ovvero qualcosa del tipo msg(MSGID, MSGTYPE, SENDER, RECEIVER, CONTENT, SEQNUM). Andando a discriminare in base al contenuto del messaggio posso decidere quali operazioni far eseguire a OutInWs. In particolare, i tipi di messaggi che il client può inviare (con relative azioni da parte del server):

- msg(eval, dispatch, SENDER, lifectl, start, 0), inviato quando il cliente clicca sul bottone "START" e farà partire il gioco, ovvero il server chiamerà la funzione *onStart()* di LifeController
- msg(eval, dispatch, SENDER, lifectl, stop, 0), inviato quando il cliente clicca sul bottone "STOP" e farà fermare il gioco, ovvero il server chiamerà la funzione *onStop()* di LifeController
- msg(eval, dispatch, SENDER, lifectl, clear, 0), inviato quando il cliente clicca sul bottone "CLEAR" e pulirà la griglia il gioco, ovvero il server chiamerà la funzione *onClear()* di LifeController
- msg(eval, dispatch, SENDER, lifectl, exit, 0), inviato quando il cliente clicca sul bottone "EXIT" e farà interrompere la connessione, ovvero il server chiamerà la propria funzione *close()*
- msg(eval, dispatch, SENDER, lifectl, cell(x,y), 0), inviato quando il cliente clicca su una cella (riporterà quindi x e y relativi) e ne cambierà lo stato, ovvero il server chiamerà la funzione *switchCellStatus(x,y)* di LifeController

Il server potrà inviare messaggi al client per comunicare la nuova configurazione della griglia o altri messaggi attraverso i metodi display.

Distribuendo via server Web la GUI del gioco si pone il problema della gestione di più utenti contemporaneamente. In base ai requisiti solo il primo utente che si connette deve essere in grado di interagire e modificare la griglia, mentre tutti gli altri sono passivi e possono solo osservare l'evoluzione. Per fare ciò, conservo lato server il primo che si connette e permetto solo a lui di interagire con il LifeController, tutti gli altri vedranno solo l'evoluzione del gioco senza poter interagire (se provano ad interagire riceveranno un messaggio "Non sei l'owner!").

In base a queste osservazioni ho sviluppato il seguente [codice per OutInWs](#).

Test plans

Project

Testing

Deployment

Il deployment avviene tramite Docker, grazie ai file [conway26JavaGuiHtml.yaml](#) e [Dockerfile](#).

Maintenance